



武汉大学
Wuhan University

第三章 Verilog HDL 基本语法

武汉大学计算机学院 2025-2026第二学期



主要内容

Main Content

Quartus II 使用说明

Verilog HDL程序基本结构

数据类型

运算符

基本语句

模块化程序设计

一、Quartus II简介

- Quartus II是Altera公司推出的综合性EDA开发软件，支持原理图和多种硬件描述语言输入形式，内嵌自有的综合器以及仿真器，可以完成从设计输入到硬件配置的完整PLD设计流程。
- Quartus II可以在Windows、Linux和Unix等操作系统中使用，为用户提供了完善的图形界面设计方式。

二、Quartus II软件界面

菜单栏

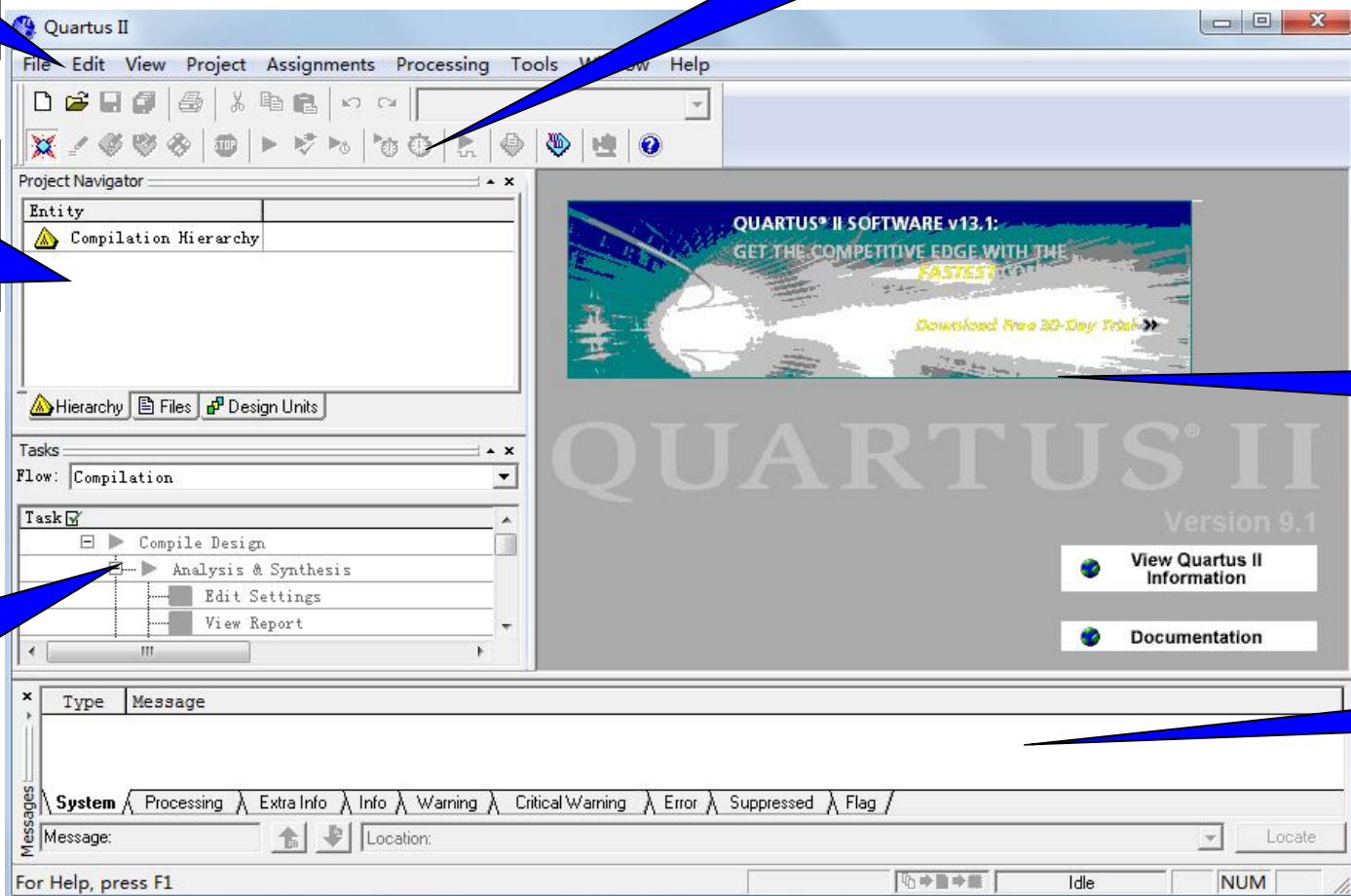
资源管理窗口

任务管理窗口

快捷工具栏

工作区

信息栏



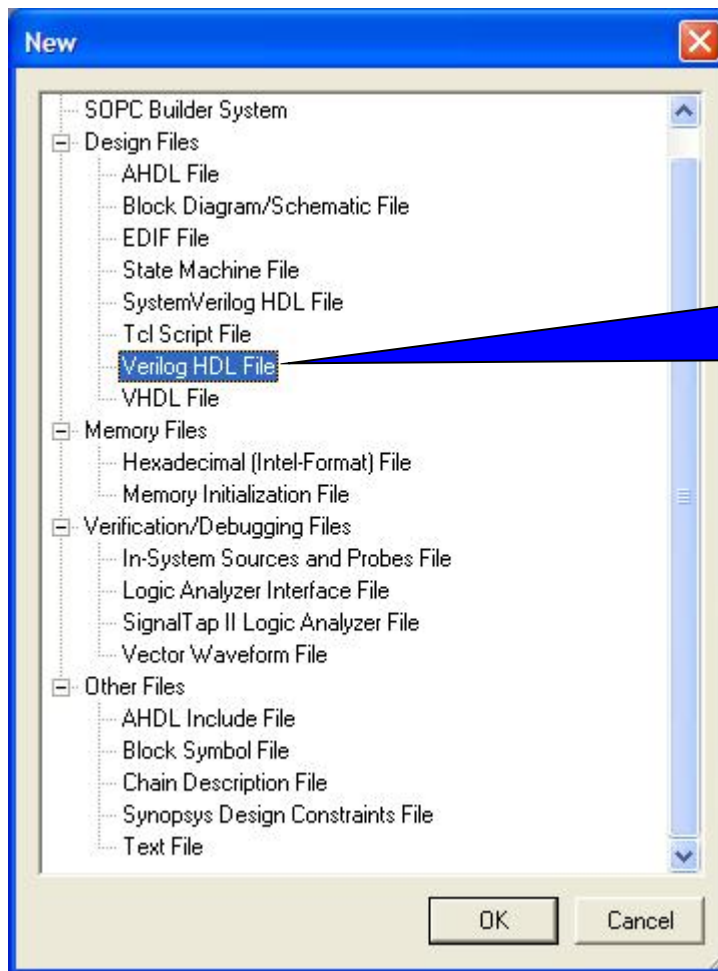
三、Quartus II开发流程

1. 新建工程

- 设置工程路径及工程名称
- 添加已有文件（若没有已有文件，则跳过）
- 选择芯片型号（若不下载到开发板上进行测试，则跳过）
- 选择仿真、综合工具（选择none，表示不用第三方工具，而是直接使用Quartus）

三、Quartus II开发流程

2. 添加文件 (file>new>Verilog HDL file)



选择以Verilog
HDL语言程序文
件作为输入

三、Quartus II开发流程

3. **编写程序代码**
4. **检查语法**
5. **分配引脚（如不下载到开发板上测试，则跳过）**
6. **编译**
7. **仿真（功能仿真、时序仿真）**
8. **下载**

主要内容

Main Content

Quartus II 使用说明

Verilog HDL程序基本结构

数据类型

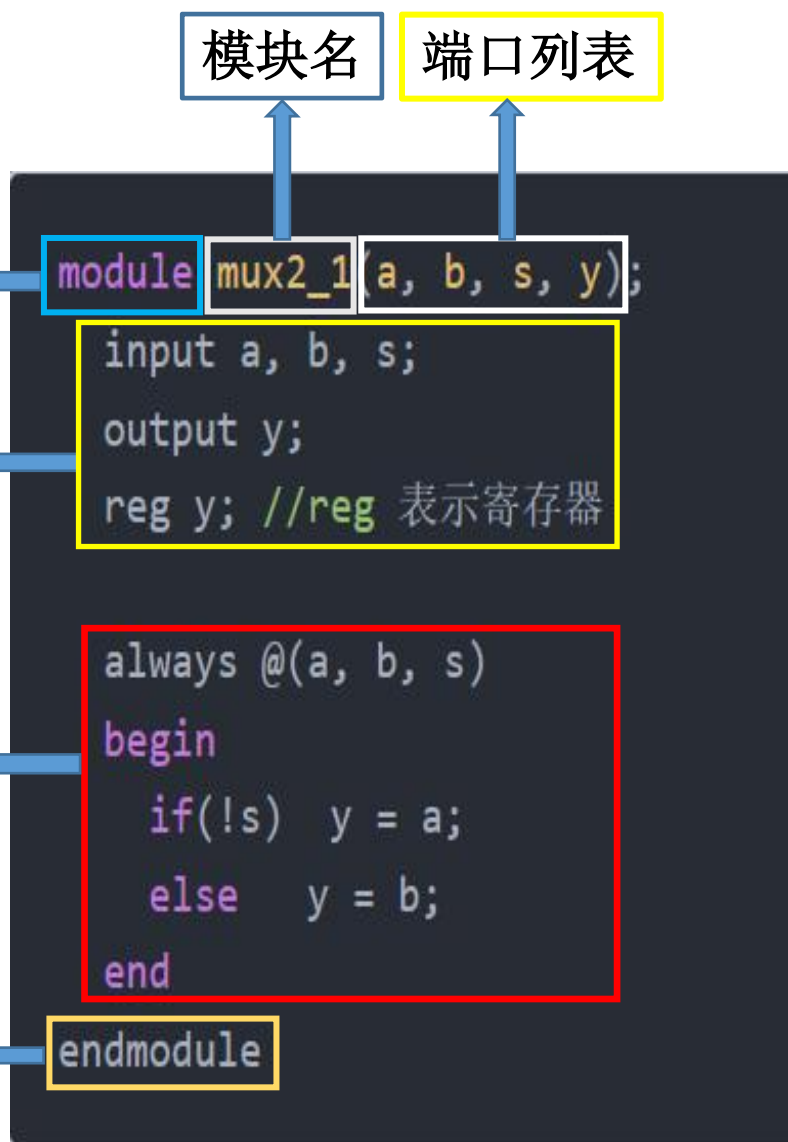
运算符

基本语句

模块化程序设计

3.1 Verilog HDL程序的基本结构

一、程序基本结构



module <模块名> (<端口列表>);

说明部分

(1)端口定义

input 输入端口

output 输出端口

端口类型说明

(2)内部信号说明

wire, reg

数据类型说明

功能描述部分

assign

begin

end

endmodule

3.1 Verilog HDL程序的基本结构

二、模块声明

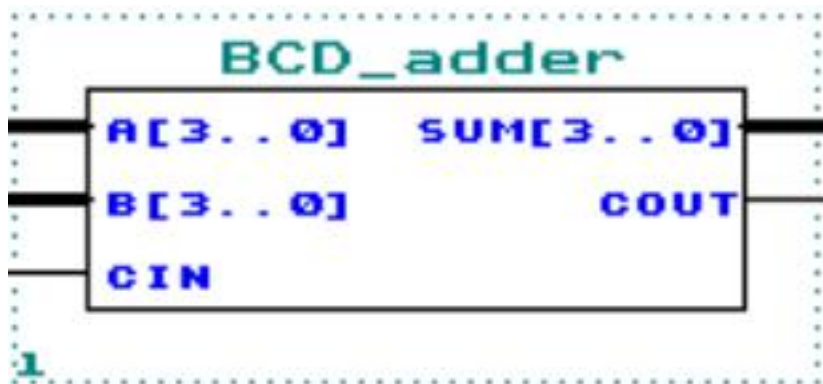
```
module 模块名(端口1, 端口2, 端口3, ...);
```

```
.....
```

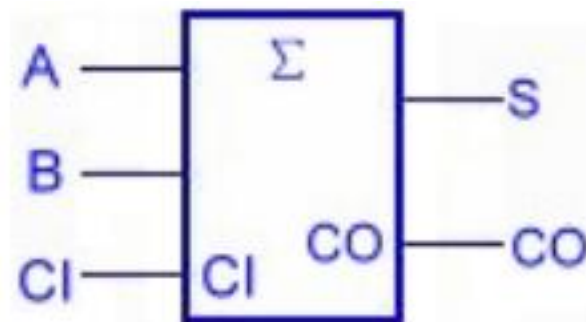
```
endmodule
```

- 模块的端口列表包括电路模块与外界联系的全部**输入/输出端口信号**，是电路模块实体对外的引脚，多个端口之间用“,”分隔
- 模块声明末尾以“;”结束

■ `module BCD_adder (A, B, CIN ,SUM, COUT);`



■ `module adder1 (A, B, CI ,S, CO);`



3.1 Verilog HDL程序的基本结构

三、模块内容

1. 端口定义

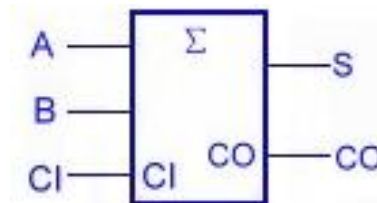
用来声明模块各端口数据流动方向

- 输入(input)
- 输出(output)
- 双向(inout)

格式(1)——信号位宽等于1位：
input/output/inout 端口1, 端口2, ...;

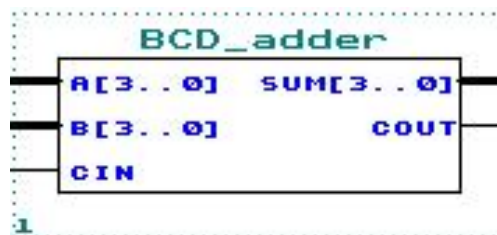
格式(2)——信号位宽大于1位：
input/output/inout [msb:lsb] 端口1, 端口2, ...;
msb-信号最高位的编号
lsb-信号最低位的编号

■ Module adder1 (A, B, CI, S, CO);



input A, B, CI;
output S, CO;

■ module BCD_adder (A, B, CIN, SUM, COUT);



input [3:0] A,B;
output [3:0] SUM;
input CIN;
output COUT;

三、模块内容

2. 数据类型说明

2.1 端口数据类型说明

- Verilog HDL中端口数据类型一般包括连线型(wire)和寄存器型(reg)。
 - ✓ wire(线网型)表示直通, 输入一发生变化, 输出马上无条件反映;
 - ✓ reg(寄存器型)表示一定要有触发条件, 输出才能反映输入的变化。
- 端口数据类型定义的实例:

```
reg [4: 1] sum;    //定义信号sum的数据类型为4位寄存器型(reg)
wire a, b, c;      //定义信号a, b, c为1位连线型(wire)
```
- 注意:
 - ✓ 输入端口和双向端口不能说明为reg型
 - ✓ 端口信号的数据类型说明缺省时, 默认为wire型

3.1 Verilog HDL程序的基本结构

2.1 端口数据类型说明

- 如果希望输出端口能保存数据，则把它声明成`reg`(寄存器型)
- 不能将input型的端口声明为`reg`型，输入端口只是反应与其相连的外部信号的变化，并不能保存这些信号值

三、模块内容

2. 数据类型说明

2.2 内部数据类型说明

➤ 常量

- ✓ 逻辑常量
- ✓ 数值常量
- ✓ 参数常量

➤ 变量

- ✓ reg型
- ✓ wire型
- ✓ Memory型
- ✓ 数字型

3.1 Verilog HDL程序的基本结构

➤ reg型

- 寄存器是数据存储单元的抽象，通过赋值语句可以改变寄存器存储的值
- 赋值语句相当于改变触发器的值
- 用来表示always模块内的指定信号，通过赋值操作符进行操作的有效变量，必须是reg型
- reg型的数据默认值是未知的
- 例： `reg[2:0] a, b, c` //定义了三个寄存器变量，每个寄存器的位宽为3

3.1 Verilog HDL程序的基本结构

➤ wire型

- 用来表示以 `assign` 关键字指定的逻辑信号
- 程序模块中的输入、输出信号类型默认为wire型
- 例： `wire[2:0] a, b, c` //定义了三条线，每条线的位宽为3

3.1 Verilog HDL程序的基本结构

三、模块内容

3. 功能描述

功能描述是Verilog HDL程序的核心部分，用于描述**模块内部结构**和**模块端口间的逻辑关系**

模块功能描述方式

结构描述方式：

对模块**内部结构**的具体描述，是抽象级别最低的描述方式

数据流描述方式：

从**数据变换**和**传送**的角度来描述设计模块

行为描述方式：

对模块**行为功能**的抽象描述

三、模块内容

3. 功能描述——结构描述方式

结构描述方式的定义

根据设计模块的逻辑电路图，调用库中的标准元器件或已设计好的模块元件，并描述元器件之间的互连，来建立逻辑电路的模型

结构描述方式的分类

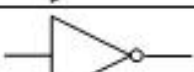
- 调用Verilog内置门元件（门级结构描述）
- 调用开关级元件（晶体管级结构描述）
- 调用用户自定义元件（模块级结构描述）

3.1 Verilog HDL程序的基本结构

三、模块内容

3. 功能描述—— 门级结构描述

Verilog
的内 置
门 元 件

类别	关键字	符号示意图	门名称
多输入门	and		与门
	nand		与非门
	or		或门
	nor		或非门
	xor		异或门
	xnor		异或非门
多输出门	buf		缓冲器
	not		非门
三态门	bufif1		高电平使能三态缓冲器
	bufif0		低电平使能三态缓冲器
	notif1		高电平使能三态非门
	notif0		低电平使能三态非门

三、模块内容

3. 功能描述——门级结构描述

门元件的调用

格式：

门元件名 <实例名> (端口列表)

实例名，是为本次元件调用后生成的门级元件实例所取的名字，可省略

➤ **普通门**的端口列表按如下顺序列出：

(输出, 输入1, 输入2, 输入3……)；

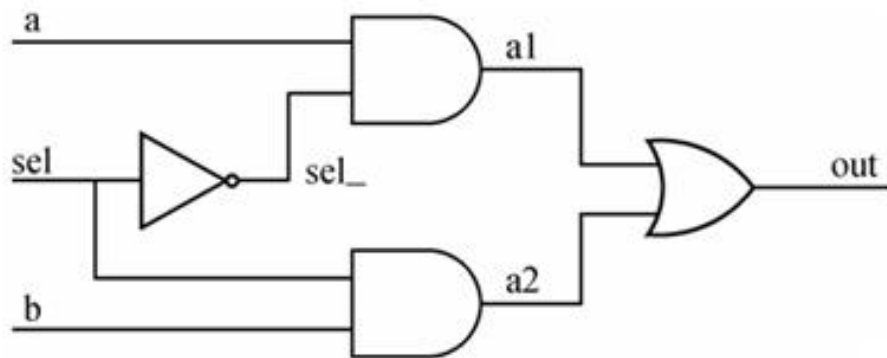
例如： **and** **a1**(out,in1,in2,in3); //三输入与门

3.1 Verilog HDL程序的基本结构

三、模块内容

3. 功能描述——门级结构描述

例如：用门级结构描述方式描述以下电路



$$\text{out} = \overline{A}C + BC$$

门级结构描述的2选1MUX

```
module MUX1(out, a, b, sel);  
    output out;  
    input a, b, sel;  
    not (sel_, sel);  
    and (a1, a, sel_);  
    and (a2, b, sel);  
    or (out, a1, a2);  
endmodule
```

三、模块内容

3. 功能描述——数据流描述方式

数据流描述方式主要使用**持续赋值语句**`assign`，多用于描述组合逻辑电路。

数据流描述方式的格式：

```
assign L_wire=R_expression;
```

说明：

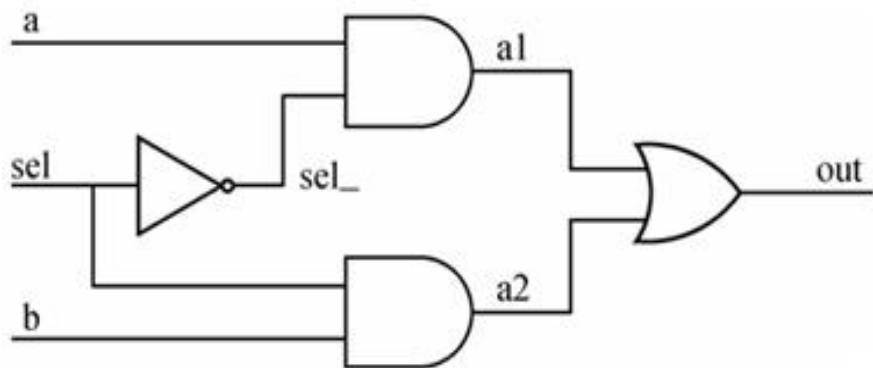
- `assign`语句的功能是将等号右侧表达式的值赋给左边的**连线型**（`wire`）变量。
- 右边表达式中的操作数**无论何时发生变化**，都会引起表达式值的**重新计算**，并将重新计算后的值赋予左边表达式的`wire`型变量。

3.1 Verilog HDL程序的基本结构

三、模块内容

3. 功能描述——数据流描述方式

例如：用数据流描述方式描述以下电路



数据流描述的2选1MUX

```
module MUX1(out, a, b, sel);
    wire out;
    output out;
    input a, b, sel;
    assign out = (a&&!sel) || (b&&sel);
endmodule
```

数据流描述方式的实现：

将系统的逻辑表达式中的逻辑运算符转换成Verilog HDL的逻辑运算符即可。

$$f = ab + \overline{c}d \quad \longrightarrow \quad \text{assign } f = (a\&\&b) \ || \ (! (c\&\&d))$$

三、模块内容

3. 功能描述——行为描述方式

- 行为描述是对设计实体**数学模型**的描述，其**抽象**程度远远高于结构描述方式和数据流方式。
- 行为描述类似于**高级编程语言**，当描述一个设计实体的行为时，无需知道具体电路的结构，只需要描述清楚**输入与输出信号的行为**，而不需要花费更多的精力关注设计功能的门级实现。
- 当电路的规模较大或时序关系较复杂时，通常采用行为描述方式进行设计。
- Verilog HDL行为描述就是在过程块**always**中，使用各种**行为语句**描述逻辑功能。

三、模块内容

3. 功能描述——行为描述方式

行为描述高级语句

- ◆ 过程块语句（initial、always）
- ◆ 块语句（begin-end、fork-join）
- ◆ 赋值语句（assign、=、<=）
- ◆ 条件语句（if-else、case）
- ◆ 循环语句（for、forever、repeat、while）
- ◆ 编译指示语句（'define、'include、'ifdef、'else、'endif）
- ◆ 任务（task）与 函数（function）

3.1 Verilog HDL程序的基本结构

小结:

- Verilog HDL程序由**模块(module)**构成，每个模块的内容嵌套在module和endmodule两语句之间；每个模块实现特定的功能，模块可以进行层次嵌套
- 每个模块首先要进行**端口定义**，并进行**I/O声明和信号类型声明**，然后**对模块的功能进行逻辑描述(结构描述、数据流描述、行为描述)**
- 除了end或以end开头的关键字（如endmodule）语句外，每条语句后必须要有分号“;”
- 可以用**/*.....*/**或**//.....**对Verilog HDL程序的任何部分作**注释**。一个完整的源程序都应当加上必要的注释，以加强程序的可读性

3.1 Verilog HDL程序的基本结构

小结:

Verilog HDL模块的 基本设计 模板

```
module <模块名> (<模块端口列表>);  
/* 端口声明*/  
    output 输出端口列表;  
    input 输入端口列表;  
/* 数据类型说明*/  
    wire 信号名;  
    reg 信号名;  
    parameter <标识符>=<表达式>;  
/*逻辑功能定义*/  
//采用assign语句定义逻辑功能  
    assign <信号名>=<表达式>;  
//调用内置门元件  
    门元件名 <实例名> (<信号顺序连接表>);  
//用过程块描述逻辑功能  
    always @(<敏感信号列表>)  
    begin  
        //阻塞赋值语句  
        //非阻塞赋值语句  
        //if-else 语句  
        //for循环语句  
        ...  
    end  
endmodule
```

主要内容

Main Content

Quartus II 使用说明

Verilog HDL程序基本结构

数据类型

运算符

基本语句

模块化程序设计

3.2 Verilog HDL的数据类型

一、常量

整数型

十进制数的形式的表示方法:表示有符号常量
例如: 30、-2

带基数的形式的表示方法:

格式为: $\langle + / - \rangle \langle \text{位宽} \rangle' \langle \text{基数符号} \rangle \langle \text{数值} \rangle$

实数型

十进制记数法 如: 0.1、2.0、5.67

科学记数法 如: 23_5.1e2, 5E-4,
23510.0, 0.0005

字符串

用“”括起来,按ASCII码序列存放。

```
reg message[1:8*12];  
message="Hello World!";
```

常量

一、常量

1. 整型常量

格式：<位宽>'<进制><数值>

◆位宽：对应的二进制宽度

◆进制：

- 二进制：b/B
- 十进制：d/D
- 八进制：o/O
- 十六进制：h/H

例如：

8'b10110001 //表示位宽为8位的二进制数

8'hf5 //表示位宽为8位的十六进制数

十进制数的位宽和进制符号可以缺省，

例如：

125 //表示十进制数125

一、常量

2. 实型常量

- 可用十进制计数法和科学计数法表示
- 小数点两边都必须有数字

例如:

7.56 //十进制计数法

3.4E3 //科学计数法

3. 字符串常量

字符串是用双引号括起来的可打印字符序列，必须包含在同一行中

例如:

“ABC”

“A BOY.”

“1234”

一、常量

4. 参数常量

- **parameter**用于定义符号常量，可增加程序的可读性及可维护性

例如：

```
parameter PI = 3.14, sign = “Hello World”;
```


二、变量

Verilog常用变量类型

类型	功能说明
wire	连线类型，表示结构实体之间的物理连接
reg	常用的寄存器型变量，被看做无符号数
integer	32位带符号整数型变量
real	64位带符号实数型变量
time	无符号时间型变量

- **wire**用来描述电路中的物理连接，没有状态保持能力，输出随着输入变化而变化
- **reg**是物理电路中数据存储单元的抽象，对应数字电路中具有状态保持作用的元件，如触发器、寄存器等。其特点是：具有记忆功能，必须明确赋值才能改变其状态，否则一直保持上一次的赋值状态
- **Integer**也是寄存器型变量，但是不允许说明位宽，具体位数取决于机器字长

主要内容

Main Content

Quartus II 使用说明

Verilog HDL程序基本结构

数据类型

运算符

基本语句

模块化程序设计

一、算术运算符

分类	操作符及功能	简要说明
算术操作符	+ - * / % 加 减 乘 除 整除	二元操作符，即有两个操作数。 %是求余操作符，在两个整数相除基础上，取余数。 例如，5%6的值是5；13%5的值是3。

二、逻辑运算符

分类	操作符及功能	简要说明
逻辑 操作符	&& 逻辑与 逻辑或 ! 逻辑非	&& 和 为二元操作符。 ! 为一元操作符，即只有一个操作数。

三、比较运算符

比 较 操 作 符	>	大于	关系运算是二元操作符， 关系运算的结果是1位逻辑值 。 如果操作数之间的关系成立，返回值为1；关系不成立， 则返回值为0。 若某一个操作数的值不定，则关系是模糊的，返回值是 不定值X。	
	<	小于		
	>=	大于等于		
	<=	小于等于		
			相等与全等操作符的区别：当操作数的某些位存在不定 值或高阻态时，相等运算的结果为不定值。全等运算当 两个操作数<u>完全一致</u>时，无论其中是否有不定态或高阻 态，比较结果为1。 例如： A=8'b1101xx01 B=8'b1101xx01 则 A==B 运算结果为 x （未知）； A===B 运算结果为 1 （真）。	
	==	相等		
	!=	不相等		
	===	全等		
	!==	非全等		

四、位运算符

分类	操作符及功能	简要说明
位 操 作 符	~ 按位非 & 按位与 按位或 ^ 按位异或 ^ ~ (~ ^) 按位同或	位运算是将两个操作数按对应位进行逻辑操作。 “~”是一元操作符，其余都是二元操作符。 将操作数按位进行逻辑运算。 例如： A=8'b11010001 ~A=8'b00101110 B=8'b00011001 A&B=8b'00010001

五、移位运算符

分类	操作符及功能	简要说明
移位 操作符	>> 右移 << 左移	二元操作符，对左侧的操作数进行右侧操作数指明的位数的移位，空出的位用0补全。 例如：设A=8'b11010001 则A>>4 结果A=8'b00001101 而A<<4 结果A=8'b00010000。

六、条件运算符

分类	操作符及功能	简要说明
条件操作符	?: 操作数=条件? 表达式1: 表达式2; 当条件为真（值为1）时， 操作数=表达式1; 当条件为假（值为0）时， 操作数=表达式2。	三元操作符，即条件操作符有三个操作数。 例如 d=a? b: c 若条件操作数a是逻辑1，则将操作数b 赋给变量d; 若a是逻辑0，则将操作数c赋给变量d

七、并接运算符

分类	操作符及功能	简要说明
并接操作符	{, }	将两个或两个以上用逗号分隔的表达式按位连接在一起 例如： X={a[7:4],b[3],c[2:0]} X由a的第7-4位、b的第3位和c的第2-0位拼接而成

八、运算符的优先级

优先级序号	操作符	操作符名称
1	!、~	逻辑非、按位取反
2	*/、/、%	乘、除、求余
3	+、-	加、减
4	<<、>>	左移、右移
5	<、<=、>、>=	小于、小于等于、大于、大于等于
6	==、!=、 ===、!==	等于、不等于、全等、不全等
7	&、~&	按位与、按位与非
8	^、^~	按位异或、按位同或
9	、~	按位或、按位或非
1 0	&&	逻辑与
1 1		逻辑或
1 2	?:	条件操作符

主要内容

Main Content

Quartus II 使用说明

Verilog HDL程序基本结构

数据类型

运算符

基本语句

模块化程序设计

3.4 Verilog HDL的基本语句

Verilog HDL的行为描述语句

用于描述电路的逻辑行为，以实现电路建模。

类别	语句	可综合性
过程块语句	initial	
	always	√
块语句	串行块begin-end	√
	并行块fork-join	
赋值语句	连续赋值assign	√
	过程赋值=、<=	√
条件语句	if-else	√
	case	√
循环语句	for	√
	repeat	√
	while	
	forever	
编译向导语句	'define	√
	'include	
	'ifdef, 'else, 'endif	√
任务和函数	task	√
	function	√

综合：将输入的Verilog HDL电路描述转化为一系
列物理单元（如逻辑门）以及这些单元之间的互
连。

不具有可综合性的语句一般用于仿
真测试。

一、过程块

Verilog HDL的行为描述以**过程块**为基本组成单位，一个模块（module）的行为描述由**一个或多个并行运行的过程块**组成。

➤ initial

initial过程块中的语句**仅执行一次**，常用于仿真中的初始化，给电路中的reg或者memory单元赋初值。

➤ always

always块内的语句是**不断重复执行**的。

一、过程块

always过程块

```
always @(敏感信号表达式event-expression)
```

```
begin
```

```
    //过程赋值
```

```
    //if-else, case, casex, casez选择语句
```

```
    //while, repeat, for循环
```

```
    //task, function调用
```

```
end
```

“always” 过程语句通常是带有**触发条件**的，触发条件写在敏感信号表达式中，只有当**触发条件满足时**，其后的“begin-end”块语句**才能被执行**

一、过程块

敏感信号表达式

敏感信号表达式又称事件表达式或敏感信号列表，即当该表达式中**变量的值改变**时，就会引发块内语句的执行。因此**敏感信号表达式中应列出影响块内取值的所有信号**。若有两个或两个以上信号时，它们之间用“or”连接。

```
always @(a) //当信号a的值发生改变
always @(a or b) //当信号a或信号b的值发生改变
always @(posedge clock) //当clock 的上升沿到来时
always @(negedge clock) //当clock 的下降沿到来时
always @(posedge clk or negedge reset)
//当clk的上升沿到来或reset信号的下降沿到来
```

3.4 Verilog HDL的基本语句

一、过程块

选择控制信号

举例：4选1数据选择器

```
module mux4_1(out, in0, in1, in2, in3, sel);  
    output out;  
    input in0, in1, in2, in3;  
    input[1:0] sel;  
    reg out;  
    always @(in0, in1, in2, in3, sel) //敏感信号列表  
    begin  
        case(sel)  
            2'b00: out=in0;  
            2'b01: out=in1;  
            2'b10: out=in2;  
            2'b11: out=in3;  
            default: out=2'bx;  
        endcase  
    end  
endmodule
```

- 过程块内有多条语句时，必须写在begin...end块中
- Begin...end块内的语句按顺序执行

二、赋值语句

1. 连续赋值语句

格式: `assign #(延时量) wire型变量名=赋值表达式`

◆含义: 只要等号右边赋值表达式中变量发生变化, 就重新计算表达式的值, 并且在指定的延时之后将新的值赋给等号左边的变量

◆连续赋值语句只能对`wire`型变量进行操作

◆连续赋值语句不能用在`always`过程块中

例如:

```
wire y, a, b, c, d;
```

```
assign #1 y=a&b&c&d;
```

//只要a,b,c,d中有变量发生变化, 就重新计算a&b&c&d, 并在1
//个单位的时间延迟后, 将计算结果赋值给y

二、赋值语句

2. 过程赋值语句

(1) 阻塞赋值

格式：赋值变量=表达式；

(2) 非阻塞赋值

格式：赋值变量<=表达式；

◆过程赋值语句用在initial或always过程块内

◆用于对reg型、memory型、integer型、time型和real型变量进行赋值

区别：

➤阻塞赋值语句在执行时，只有当一条语句执行结束后，才能执行下一条语句，相当于“串行”

➤非阻塞赋值语句在执行时，各条语句同时执行，相当于“并行”

3.4 Verilog HDL的基本语句

//阻塞赋值

```
module block(c,b,a,clk);  
    output c,b;  
    input clk,a;  
    reg c,b;  
    always @(posedge clk)  
        begin  
            b=a;  
            c=b;  
        end  
endmodule
```

执行结果:

a=b=c, 都等于程序执行之前a的值

//非阻塞赋值

```
module non_block(c,b,a,clk);  
    output c,b;  
    input clk,a;  
    reg c,b;  
    always @(posedge clk)  
        begin  
            b<=a;  
            c<=b;  
        end  
endmodule
```

执行结果:

**b=程序执行前a的值
c=程序执行前b的值**

三、条件语句

1. if...else语句

格式（1）：**if** (条件表达式) 语句;

格式（2）：**if** (条件表达式) 语句1;
else 语句2;

格式（3）：**if** (条件表达式1) 语句1;
else if (条件表达式2) 语句2;
else if (条件表达式3) 语句3;
...
else 语句n;

➤ **if**和**else**后面的语句可以是单条语句，也可以是由多条语句组成的**语句块**

➤ 语句块的格式如下：

```
begin  
    语句1;  
    语句2;  
    ...  
end
```

三、条件语句

2. case语句

格式:

```
case (条件表达式)
    分支表达式1: 语句1;
    分支表达式2: 语句2;
    ...
    分支表达式n: 语句n;
    default: 语句m;
endcase
```

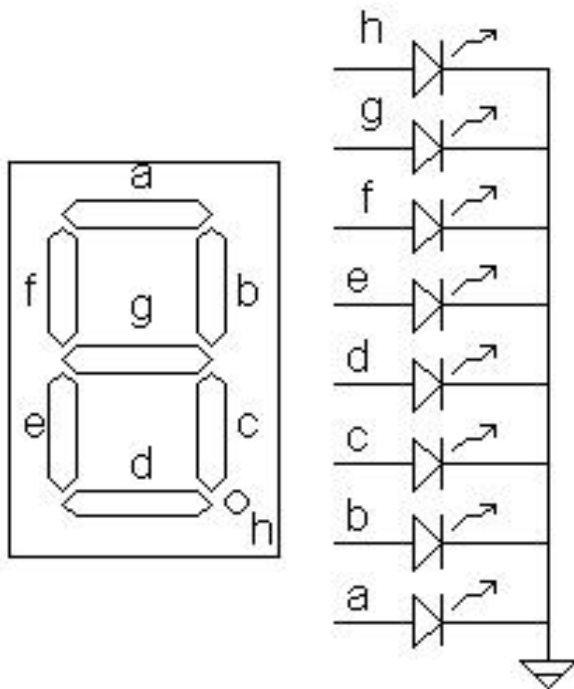
Default 覆盖
所有没有被分
支表达式覆盖
的语句。

3.4 Verilog HDL的基本语句

三、条件语句

2. case语句

7段数码管
显示译码器



```

module    decode4_7(decodeout, indec) ;
    output[6:0]  decodeout;
    input[3:0]   indec;
    reg[6:0]    decodeout;
    always @ (indec)
        case (indec) //用case语句进行译码
            4'd0: decodeout=7'b1111110;
            4'd1: decodeout=7'b0110000;
            4'd2: decodeout=7'b1101101;
            4'd3: decodeout=7'b1111001;
            4'd4: decodeout=7'b0110011;
            4'd5: decodeout=7'b1011011;
            4'd6: decodeout=7'b1011111;
            4'd7: decodeout=7'b1110000;
            4'd8: decodeout=7'b1111111;
            4'd9: decodeout=7'b1111011;
            default: decodeout=7'bx;
        endcase
    endmodule
    
```

第3章习题 P106-107

1、4、8、9



武汉大学
Wuhan University

谢谢!

2025-11-20

